

CS24300 PROJECT-I

***HELIOS: An Alpha Driven Quantitative Trading System with
Fault Tolerant and Low Latency Infrastructure***

SRS + SDS Report

BY

Abhishek Kumar BTECH/25002/24

Anuj Sharma BTECH/25011/24

Indra Shikari BTECH/25028/24

Under the supervision of

Dr. Shripal Vijayvargiya

Head of Department



**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
BIRLA INSTITUTE OF TECHNOLOGY, MESRA
JAIPUR CAMPUS, JAIPUR
MO-2026**

1. Introduction

1.1 Purpose

In quantitative trading, the rule used to decide when to buy and when to sell is written as a formula or as a model. This rule is called an alpha. An alpha is simply a number that tells us what the model believes will happen next in the market.

The purpose of this project is to build a system that creates such alphas, tests them on old market data, and finds out whether they would actually have made money. The system does not stop at the alpha. It converts the alpha into buy and sell orders, checks those orders against risk rules, sends them to a simulated exchange and reports the profit or loss.

No real money is used. All trading is paper trading, which means orders are executed inside our own software and nothing goes to a real exchange. The project also has two supporting parts: making the system keep working when a machine fails, and measuring how fast the system actually is instead of only claiming that it is fast.

1.2 Scope

Many projects in this area train a model to predict whether the price will go up or down and then report the accuracy of that model. The problem is that a model with good accuracy can still lose money, because the profit from a correct prediction can be smaller than the cost of trading. Accuracy and profit are not the same thing. Our project fills this gap by judging every alpha using trading measures such as Sharpe ratio, turnover, drawdown and profit after cost.

The system will do the following work:

- Read historical order book data and rebuild the order book from it.
- Calculate features such as order book imbalance, spread, microprice and volatility.
- Create alphas in two ways: by writing formulas, and by training a machine learning model.
- Test every alpha in a simulator, keep the ones that pass our criteria and reject the rest.
- Run a full backtest on the selected alpha with trading costs included.
- Turn the alpha into continuous buy and sell quotes, check each order in a separate risk module, and execute it in the paper trading module.
- Run the matching engine on three machines so that it survives a machine failure, and measure the delay at each stage.

The expected outcome is a working pipeline, a record of every alpha we tried, at least one alpha that passes all criteria on data it has never seen, a backtest report with profit and risk figures, a matching engine that survives a node failure, and a table of measured delays. If no alpha passes, we will report that honestly, because a negative result is still a result. The application area is finance, mainly electronic trading and market microstructure.

1.3 Motivation

Three things pushed us towards this problem.

1. Published work on order book prediction reports classification accuracy, but very few papers connect that accuracy to actual trading profit. We wanted to build the missing part.
2. Real trading systems are not only models. They also have risk checks, order management and execution logic, and these parts decide whether a good idea survives in the market. They are rarely covered in academic projects.
3. Trading systems must not lose orders when a machine fails and must respond quickly. These are classic distributed systems problems, and they let us combine finance with our core computer science subjects.

1.4 Technology and Tools Used

The dataset is the most important item below. Without message level order book data the main part of the project cannot be done, so arranging the data is our first task.

Item	Choice	Where it is used
Languages	C++, Python, Go or Rust	C++ for the fast path, Python for research, Go or Rust for cluster services
ML library	PyTorch, PyTorch Geometric	Training the graph based model
Formal tool	TLA+ with TLC	Checking the replication protocol
Storage	Kafka or Redpanda, PostgreSQL, Redis type store	Event log, alpha and order records, live order book
Monitoring	Prometheus, Grafana	Metrics and dashboards
Deployment	Docker, Kubernetes	Running the services
Dataset	Message level order book data (LOBSTER or similar), and our own synthetic generator	Main input to the pipeline, and backup data for stress testing

2. Literature Review

2.1 Summary of existing work

DeepLOB used convolution layers followed by an LSTM to predict short term price movement from order book snapshots. HLOB added a structure based on the hierarchy of price levels, TLOB used a transformer with attention over both time and price levels, and LiT is a recent model that focuses on learning a representation that transfers from one stock to another. All of these are trained with labels, which means someone has to mark whether the price went up or down after each snapshot, and getting a large amount of such data is difficult.

For generating order flow when real data is not available, simulated exchanges are used. Hawkes processes are common for this because real order arrivals come in bursts and a Hawkes process naturally produces bursts. Cont and de Larrard described the statistical properties that a good order book simulator should reproduce.

For keeping several copies of the same state in agreement, Paxos and Raft are the standard protocols. Both apply a command only after a majority of machines have agreed on it. Some later protocols reduce the delay by doing the work early and undoing it if agreement fails.

2.2 Gaps we found

- Prediction accuracy is reported, but profit after trading cost usually is not.
- Most models need a lot of labelled data, and label efficiency is rarely tested.
- The published ablation study of TLOB itself showed that a plain MLP could match its performance on one dataset. Complicated models are therefore not always doing more than simple ones.
- Market manipulation is hard to study because real data does not tell us which orders were manipulative. There is no public dataset with correct labels.
- General purpose replication protocols are not designed for a matching engine, where one machine writes to one symbol at a time and the matching rule is fully deterministic.

2.3 How our work is different

We do not claim that the individual ideas here are new. Existing work has focused mainly on prediction accuracy, and comparatively less attention has been given to whether those predictions survive as trading strategies after costs. That connection is what our project tries to build. Our differences are listed below.

1. Every alpha, whether it comes from a formula or from a model, is judged by the same trading measures and not by accuracy.
2. Our model is trained without labels first, using two self supervised tasks, and then tested to see how few labels it needs afterwards.
3. We build a simulator that injects manipulation patterns we created ourselves, so the correct labels are known by construction.
4. Our replication protocol uses the fact that matching is deterministic and single writer, which general protocols cannot assume.

3. Research Requirements (SRS Part)

3.1 Functional Requirements

3.1.1 Data collection and preprocessing

- The system shall read message level order book data and convert it into one common event format.
- The system shall rebuild the full order book and keep price and time priority.
- The system shall detect missing, duplicate and out of order messages and report them. It shall not fill the missing part by guessing.
- Features shall be calculated only from data available up to that moment, and all scaling shall use past data only. Using the mean of the whole dataset is not allowed, because the whole dataset includes the future.

3.1.2 Alpha generation, training and validation

- The system shall support alphas written as formulas and alphas produced by a trained model.
- Every alpha shall be saved with a version number, its formula or model reference, its horizon and its author, and shall subtract the expected trading cost before it is treated as a signal.
- The data shall be split by time into training, validation and test parts, with a gap between them at least as long as the prediction horizon. Random splitting is not allowed for time series data.
- The model shall be trained only on the training split, including the unlabelled pre-training. The test split shall be used at most twice and every use shall be recorded.

3.1.3 Evaluation

- The system shall calculate Sharpe ratio, Fitness, turnover, profit and loss, drawdown, volatility and hit rate for every alpha, both before and after trading costs.
- The system shall apply the selection criteria automatically and record a pass or fail for each one.
- Rejected alphas shall also be stored, along with the reason for rejection.

The selection criteria are given below. These are targets an alpha has to meet before we accept it. They are not results, and we are not promising that any alpha will meet them.

Measure	Condition	Reason
Sharpe ratio	Greater than 1	Return should be reasonable compared to the risk taken
Fitness	Greater than 1	Return should not depend on trading a very large amount
Turnover	Between 1% and 70%	Below 1% the alpha does almost nothing, above 70% the cost eats the profit

3.1.4 Strategy, risk and execution

- The strategy shall keep a buy quote and a sell quote in the market at the same time and update them as conditions change. It shall not place a single buy order and a single sell order.
- The strategy shall change its quotes based on the position it is holding, so that the position is pulled back towards zero.
- Every order shall pass through the risk module before it is sent, and there shall be no way to skip this. The risk module shall check order size, price band, position limit, exposure limit and order rate.
- The risk module shall have a kill switch that stops all new orders, and a circuit breaker that trips automatically when the order rate or loss becomes abnormal.
- The order module shall track each order through its full life, and profit and loss shall be calculated only when a fill is received, never when an order is sent.

3.1.5 Fault tolerance

- The matching engine shall match orders continuously across several price levels, in price and time order, and shall allow partial fills against several counterparties.
- The engine state shall be copied to more than one machine, and a command shall be final only after a majority agree.
- If agreement fails, the machine shall undo the early work and rebuild by replaying the agreed commands. The result of early work shall never be shown outside the machine before agreement is reached.

- Replaying the same commands from the same starting state shall always give exactly the same result.

3.2 Non-Functional Requirements

Type	Requirement
Performance	The delay of each stage shall be measured and reported at the 50th, 95th and 99th percentile with its variation. A single average is not acceptable because it hides the worst cases.
Performance	The backtest shall finish one day of data for one instrument in a few minutes, so that many ideas can be tried.
Correctness	No agreed trade shall be lost, duplicated or reordered when a minority of machines fail, and the profit and loss account shall balance at every checkpoint.
Scalability	Adding a new instrument shall need only a configuration change, and alphas shall be testable in parallel.
Reproducibility	Every reported number shall be reproducible from the same code version, data and random seed. The same feature calculated in Python and in C++ shall give the same value within a small tolerance.
Reliability	If the risk module fails, it shall reject every order rather than allow every order.
Security	Changing a risk limit or using the kill switch shall need a specific role and shall be written to an audit log.
Hardware	A GPU is needed for model training. The fast path needs a machine where threads can be fixed to particular CPU cores.

3.3 Assumptions and Constraints

3.3.1 Assumptions

- Message level order book data will be available for a few instruments over a few weeks of trading, and a GPU will be available for training.
- Simulated execution with a clearly described fill rule is acceptable for an academic evaluation.
- Machines can crash and messages can be lost or delayed, but no machine will behave dishonestly.
- System clocks are roughly in agreement, and correctness never depends on them being exact.

3.3.2 Constraints

- No real money and no connection to a real exchange. All execution is simulated.
- We do not have space inside an exchange building, so we cannot claim colocation. We can only build a similar arrangement in the cloud and measure it.
- We do not have FPGA or special network cards, so no hardware acceleration is claimed.
- Formal checking covers the safety properties of the protocol model on small configurations. It does not prove that the actual code is correct.
- Three team members working part time along with regular coursework, on cloud machines shared with other users. Part of the delay we measure is therefore outside our control, and this will be stated with the results.

3.3.3 Points still to be decided

Open point	Needed by
Which dataset we will use, and whether the paid version is required	Immediately
Which time horizon the alpha is aimed at, since the turnover limit means different things for a one day alpha and a one second alpha	Before the first alpha is tested
The exact formula for Fitness, including its constants	Before the first alpha is accepted
Which language is used for which service, and the fill rule used in the backtest	Before coding the trading path

4. System Design (SDS Part)

4.1 Research Workflow and System Architecture

The system works as one long pipeline. Data comes in at the top and a profit and loss figure comes out at the bottom. The important point is the check in the middle. An alpha only moves forward into the trading

part if it passes the three selection criteria. If it does not pass, it is stored with its results and we move on to the next idea. The dashed line shows that the results are fed back into our research.

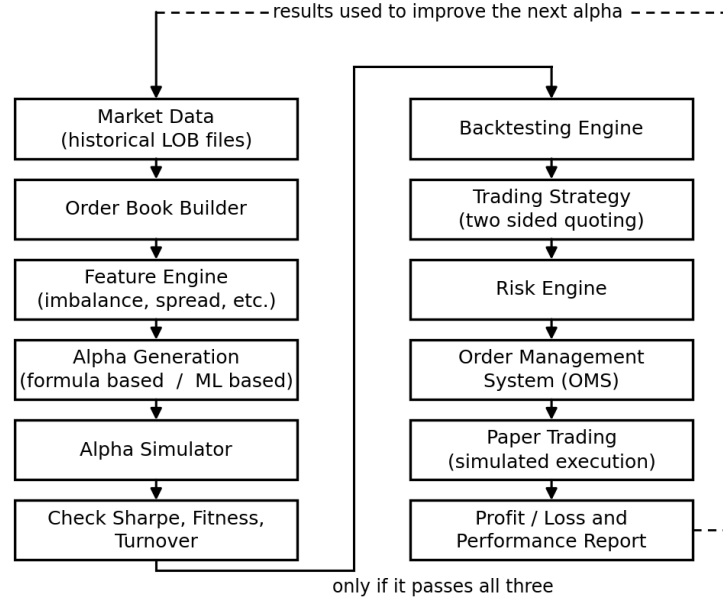


Figure 1: Overall workflow of HELIOS

At present the orders go to the paper trading module. Later the same orders will go to the fault tolerant matching engine instead. Both accept orders in exactly the same format, so we can switch between them by changing one setting and without rewriting the strategy.

4.2 Module Design

The system is divided into ten modules. Each module does one job and passes its output to the next one.

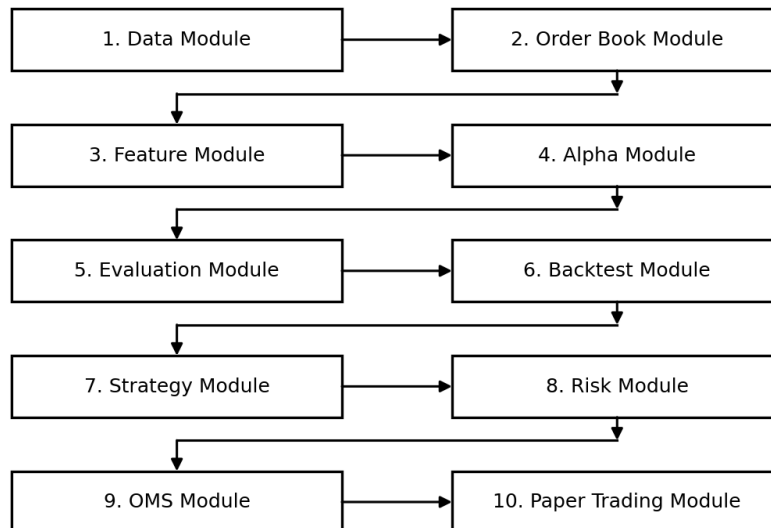


Figure 2: Module block diagram

Module	What it does
1. Data and 2. Order Book	Read the data files, convert every message into one common format, check for gaps, and build the order book with price and time priority
3. Feature and 4. Alpha	Calculate imbalance, spread, microprice and volatility, and from these calculate the alpha value using a formula or the trained model
5. Evaluation and 6. Backtest	Run the alpha through the simulator, apply the selection criteria, and replay history through the full strategy with costs and simulated fills

Module	What it does
7. Strategy and 8. Risk	Decide the buy and sell quote prices and sizes, then check every order and reject unsafe ones using the kill switch and circuit breaker
9. OMS and 10. Paper Trading	Give each order an identity and track it, decide the fills using a queue based rule, and update position and profit

Two modules are often confused, so we state the difference clearly. The Alpha Module answers the question "what do we think will happen". The Strategy Module answers the question "given that, what orders should we place". It is quite possible for the alpha to be positive while the strategy still places a lower buy price, because we are already holding a large long position and want to reduce it.

4.3 Algorithm and Model Design

4.3.1 Alpha selection procedure

The flowchart below shows how a candidate alpha is either accepted or rejected.

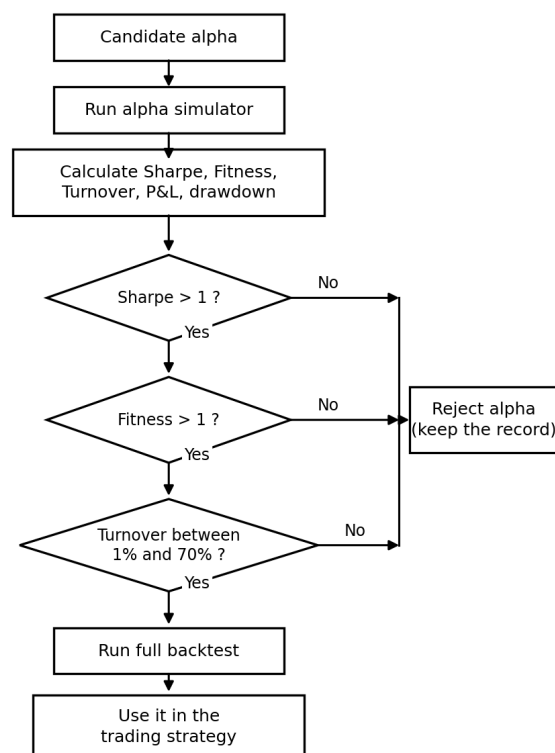


Figure 3: Alpha selection flowchart

Two checks are run on the simulator itself before we trust its output. First we feed it a random alpha, which should give a Sharpe close to zero. Then we feed it an alpha that is deliberately allowed to see the future, which should give an impossibly high Sharpe. If either test gives the wrong answer, the simulator has a bug and every result from it is meaningless.

4.3.2 Feature calculation

$$\begin{aligned}
 \text{mid price} &= (\text{best bid} + \text{best ask}) / 2 \\
 \text{spread} &= \text{best ask} - \text{best bid} \\
 \text{microprice} &= (\text{best bid} * \text{ask size} + \text{best ask} * \text{bid size}) \\
 &\quad / (\text{bid size} + \text{ask size}) \\
 \text{imbalance (k)} &= (\text{bid size}(k) - \text{ask size}(k)) \\
 &\quad / (\text{bid size}(k) + \text{ask size}(k))
 \end{aligned}$$

A small example makes the imbalance clear. Suppose the best bid is 100.02 with 800 shares and the best ask is 100.03 with 200 shares. Then the imbalance is $(800 - 200) / (800 + 200) = 0.60$. There is much more size waiting to buy than to sell, so there is pressure towards the upper side. The microprice comes out as 100.028, which is above the mid price of 100.025, and it leans towards the thin side of the book.

4.3.3 Example alpha (for illustration only)

The following shows the form of a formula based alpha. It is only an example and is not our final alpha.

```
z1    = zscore(imbalance, past window)
z2    = zscore(order flow imbalance, past window)
z3    = zscore(spread, past window)

alpha = w1*z1 + w2*z2 - w3*z3
alpha = clip(alpha, -1, +1)
alpha = alpha - expected trading cost
```

The z-score uses only past data. The clip keeps the alpha inside a fixed range so the position never becomes unlimited. The last line removes the expected trading cost, because a signal smaller than the cost of trading is not useful even if it is correct.

4.3.4 Order path and risk check

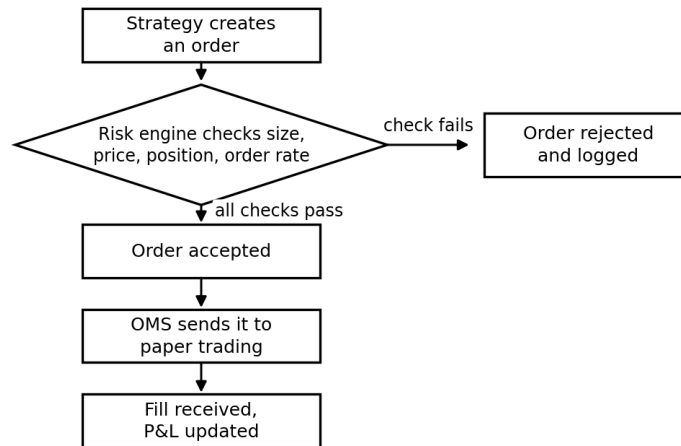


Figure 4: Path of an order from strategy to fill

The position check is done on the projected position, which is the current position plus the orders already waiting in the market plus the new order. If we checked only the current position, the strategy could send many orders that are each within the limit but together cross it when they all get filled.

4.3.5 Machine learning model

For the machine learning route we treat each order book snapshot as a small graph. Each price level is a node and neighbouring price levels are joined by edges. A graph neural network processes this graph, and a transformer with a causal mask processes the sequence of these graphs over time. The causal mask makes sure that the model at time t cannot look at time $t+1$.

The model is first trained without labels using two tasks. In the first, part of the event stream is hidden and the model has to predict it. In the second, the model learns to place snapshots with similar queue positions close to each other. We then compare the learned representation with DeepLOB, HLOB, TLOB and LiT, and separately convert the model output into an alpha and send it through the same simulator and criteria as a formula based alpha. Both results will be reported, because a better representation does not automatically mean a better trading result.

4.3.6 Replication protocol

The matching engine runs on three machines. One is the leader and the other two are followers.

1. The leader receives an order command and gives it a position number in its log.
2. The leader applies the command to its own copy of the order book straight away, but keeps the result inside itself. Nothing is sent out yet.
3. The leader sends the command to the two followers.
4. When at least two out of three machines have stored the command, it is treated as final.
5. Only now the leader sends out the trade reports.
6. If agreement is not reached, the leader throws away the early result, goes back to the last agreed state, and rebuilds by replaying the agreed commands in order.

Step 2 saves time because the matching work happens at the same time as the network round trip. Step 5 is the safety rule. If a trade result escaped in step 2 and was then thrown away in step 6, we would have reported a trade that never happened. This is the property we will write in TLA+ and check with TLC. For replay to be safe, applying the same command must always give the same answer, so the matching code cannot read the clock, cannot use random numbers, and must use whole numbers for prices and sizes instead of decimals.

4.4 Database and Storage Design

Data	Stored in	Reason
Market events	Parquet files, one folder per instrument and date	Large volume, read in bulk during research
Live order book	Memory, with a Redis type store	The fast path cannot wait for disk
Engine event log	Kafka or Redpanda	Append only and can be replayed in order
Alpha records, orders, fills, audit log	PostgreSQL	Needs relations, transactions and easy querying
Trained models and reports	Object storage	Needed for reproducing results later

The main table is the alpha record table. It stores the alpha identity and version, the type (formula or model), the formula or model reference, the horizon, the exact date ranges used for training, validation and test, the values of Sharpe, Fitness and turnover, the profit, drawdown and volatility, the trading cost assumed, the pass or fail result for each criterion, the current status, how many times the test data has been used, and the code version, data version and random seed needed to reproduce the result.

4.5 Experimental Design

4.5.1 Data split

```
|<---- TRAINING ---->|gap|<-- VALIDATION -->|gap|<-- TEST -->|
```

The data is divided by time. Random splitting is not used because the data is a time series, and shuffling it would let information from the future leak into the training set. The training part fits the model, the validation part is used to choose settings and compare alphas, and the test part is only used at the end to confirm the result. It may be used at most twice and every use is recorded. The gap between the parts is at least as long as the prediction horizon, so a label belonging to one part does not overlap into the next.

4.5.2 Settings and hardware

Item	What we will record or use
Alpha settings	Window lengths, weights, clipping limits, number of free parameters
Model settings	Number of layers, hidden size, learning rate, batch size, epochs, random seed
Cost and fill rule	Assumed cost in basis points, and which fill rule was used: optimistic, queue based or pessimistic
Delay setting	The decision delay added in the backtest, so the strategy is not treated as infinitely fast
Hardware	One GPU machine with at least 16 GB GPU memory, one machine where CPU cores can be reserved, and three machines for the cluster

4.5.3 What we will measure

The table below lists what will be measured. None of these numbers exist yet, so the last column is left as "to be measured" and will be filled in only after the experiment is actually run.

Area	Measure	Result
Alpha	Sharpe, Fitness and turnover on test data, and Sharpe at higher assumed cost	To be measured
Trading and risk	Profit and loss, drawdown, fill rate, orders rejected, kill switch time	To be measured
Matching engine	Property tests and invariant violations	To be measured

Area	Measure	Result
Replication	Agreement delay at 50th, 95th and 99th percentile compared with Raft, and recovery time under injected faults	To be measured
Model	Comparison with DeepLOB, HLOB, TLOB and LiT, and accuracy with 1%, 5%, 10%, 25% and 100% of the labels	To be measured
Delay	Delay at each stage at 50th, 95th and 99th percentile	To be measured

4.5.4 Plan of work

Stage	Work	Finished when
1	Data, order book, features	Order book rebuilt correctly for a full day of data
2	Alpha, simulator, criteria, records	One alpha tested end to end with all measures
3	Backtest, strategy, risk, OMS, paper trading	An accepted alpha trades on paper with correct profit and loss
4	Machine learning model	Model trained and its output tested as an alpha
5	Matching engine, replication, TLA+, fault injection	Cluster survives a node failure and replay gives the same state
6	Delay measurement and final report	Delay table filled with measured values and results written up

5. References

- [1] Z. Zhang, S. Zohren and S. Roberts, "DeepLOB: Deep Convolutional Neural Networks for Limit Order Books," IEEE Transactions on Signal Processing, vol. 67, no. 11, pp. 3001-3012, 2019.
- [2] A. Briola, S. Bartolucci and T. Aste, "HLOB: Information Persistence and Structure in Limit Order Books," Expert Systems with Applications, 2025.
- [3] L. Prata et al., "TLOB: A Transformer Model with Dual Attention for Price Trend Prediction with Limit Order Book Data," arXiv preprint, 2025.
- [4] R. Cont and A. de Larrard, "Price Dynamics in a Markovian Limit Order Market," SIAM Journal on Financial Mathematics, vol. 4, no. 1, pp. 1-25, 2013.
- [5] M. Avellaneda and S. Stoikov, "High-frequency Trading in a Limit Order Book," Quantitative Finance, vol. 8, no. 3, pp. 217-224, 2008.
- [6] D. Ongaro and J. Ousterhout, "In Search of an Understandable Consensus Algorithm," USENIX Annual Technical Conference, pp. 305-319, 2014.
- [7] L. Lamport, "Paxos Made Simple," ACM SIGACT News, vol. 32, no. 4, pp. 51-58, 2001.
- [8] N. Hayashibara, X. Defago, R. Yared and T. Katayama, "The phi-accrual Failure Detector," IEEE Symposium on Reliable Distributed Systems, pp. 66-78, 2004.
- [9] A. G. Hawkes, "Spectra of Some Self-exciting and Mutually Exciting Point Processes," Biometrika, vol. 58, no. 1, pp. 83-90, 1971.
- [10] R. Huang and T. Polak, "LOBSTER: Limit Order Book Reconstruction System," Humboldt University Berlin, Technical Report, 2011.
- [11] A. Vaswani et al., "Attention Is All You Need," Advances in Neural Information Processing Systems, vol. 30, 2017.

Note: references 2 and 3 are recent papers and we are still verifying the full author list and page numbers for the final report.